tensormodels: Statistical Models for Tensor-Valued Data

by Anupama Kannan, David Kahle, and Michael Gallaugher

Abstract tensormodels is an R package for simulation, maximum likelihood estimation, and model diagnostics for tensor-variate distributions arising in multiway array data. The package implements the tensor-variate normal distribution and several normal variance–mean mixture families, including the skew-t, generalized hyperbolic, variance gamma, and normal inverse Gaussian distributions. Estimation is performed via likelihood-based and expectation–maximization algorithms under a Kronecker-separable covariance model, yielding mode-specific covariance estimates through iterative updates. Model assessment tools include distance-based diagnostics, such as Mahalanobis distance plots constructed from tensor-whitened residuals. In addition, tensormodels provides implementations of widely used tensor decompositions for approximation and dimension reduction, including Tensor-Train decomposition via TT-SVD, Higher-Order Singular Value Decomposition (HOSVD), and Parallel Factor Analysis (CP/PARAFAC) computed using alternating least squares. It includes common linear algebra and tensor operations, such as matricization, the n-mode products, and the Khatri-Rao product.

1 Introduction

Lately, with the rise of data measurement tools and big data, the creation of higher dimensional data has become more and more common. These unique types of data require corresponding unique analytical tools to properly fit models with interpretation. Many researchers default to vectorizing or matricizing this higher order data to fit models that they are comfortable with and have pre-established tools. Additionally, they will commonly assume that data satisfies the assumptions of normality, which can sometimes be a very extreme choice that is only made to make computation and analysis easier.

The package **tensormodels** aims to assist with both of these issues. Firstly, it contains the tensor variate normal distribution and several skewed distributions under the normal mean-variance family model that can model a multitude of different shapes of data. Secondly, it contains diagnostic test and model heuristics to pick the best distribution for the user's data set. It also contains random number generation and density functions for researchers who merely want to better understand tensor variate distributions.

In the following sections, we will explain the theory behind tensor variate distributions, explain some unique choices in the package design, show a simulation set up, and apply the models to a real-world problem.

2 Background and related work

The tensor space is still being established, with works using conflicting notation. In this section, we will set a standard for certain operations, and explain the basic underlying theory for several of the functions. Tensors are a generalization of vectors and matrices, allowing us to represent higher-order data into a structure. We need new terminology to describe these objects.

2.1 Order

The order of a tensor is the number of indices needed to specify a value. A vector has order 1 and a matrix has order 2, while the order of a tensor is 3 or more. Another way to define the order is the number of values needed in R's subsetting operator to pull out a specific element.

For example, take the vector $\mathbf{a} = (1, 2, 3)$. To extract a specific element, you only require one value, so it has an order 1.

```
a <- c(1, 2, 3)

a[2]
#> [1] 2
```

Now consider the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

In R, we would need to specify two indices to pull out an element - the first represents the row, and the second represents the column.

```
A <- matrix(data = 1:6, nrow = 2, ncol = 3)

A[2, 1]
#> [1] 2
```

Consider a third-order tensor. In R, we would need to specify three indices to pull out an element.

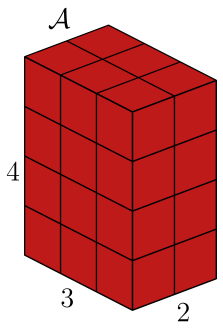


Figure 1: A $4 \times 3 \times 2$ tensor $\mathcal{A}$

```
A <- array(data = 1:24, dim = c(4, 3, 2))

A[3, 1, 1]
#> [1] 3
```

2.2 Mode

The mode of an object is its specific axis. For a matrix, it is common terminology to refer to the first mode as the rows and the second mode as the columns. However, since a tensor can have three or more dimensions, we now need a term for the specific dimension being referred to.

Consider a tensor $\mathcal{A} \in {}^{4\times2\times3}$. We would say the first mode is 4, the second mode is 2, and the third mode is 3. Note that the modes of a tensor are dependent on the structure of the object. For example, we could keep the same data in $\mathcal{A}$ but permute the modes, leading to a new tensor $\mathcal{B} \in {}^{4\times3\times2}$. When communicating a tensor, it is important to keep the modes consistent and keep track of what each mode represents.

2.3 n-mode product

The n-mode matrix product of a tensor $\mathcal{A} \in {}^{I_1\times I_2\times\ldots\times I_n}$ with a matrix $\mathbf{U} \in {}^{J\times I_n}$ is

$$(\mathcal{A} \times_n \mathbf{U}) \in {}^{I_1\times\ldots\times I_{n-1}\times J\times I_{n+1}\times\ldots\times I_N}$$

with entries

$$(\mathcal{A} \times_n \mathbf{U})_{i_1,\ldots,i_{n-1},j,i_{n+1},\ldots I_N} = \sum_{i_n=1}^{I_n} a_{i_1,i_2,\ldots,i_N} u_{j,i_n}.$$

The tensor and matrix share one mode in common, denoted here as I_n. This operation is also called the tensor times matrix product.

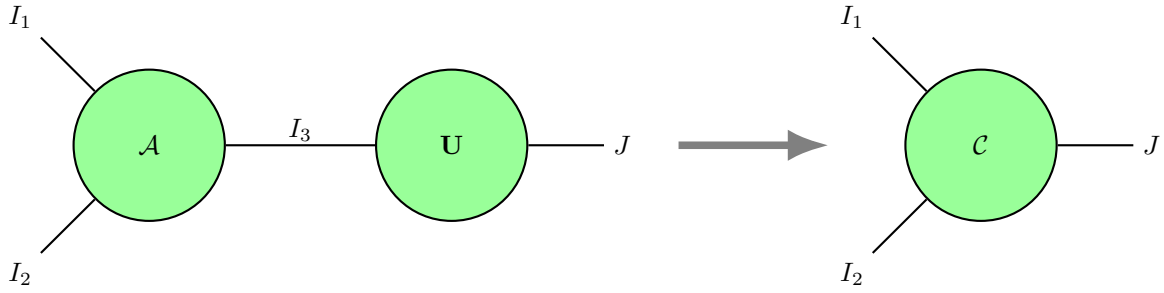


Figure 2: n-mode product

Example

Let $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \in {}^{3}$ and $\mathbf{U} = \begin{bmatrix} 4 & 6 & 8 \\ 5 & 7 & 9 \end{bmatrix} \in {}^{2\times 3}$.

$$\mathbf{a} \times_1 \mathbf{U} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \times_1 \begin{bmatrix} 4 & 6 & 8 \\ 5 & 7 & 9 \end{bmatrix} = \begin{bmatrix} 1(4)+2(6)+3(8) \\ 1(5)+2(7)+3(9) \end{bmatrix} = \begin{bmatrix} 40 \\ 46 \end{bmatrix}$$

2.4 nm-mode product

The ${}_n\times_m$-product of a tensor $\mathcal{A} \in {}^{I_1\times I_2\times\ldots\times I_N}$ with a tensor $\mathcal{B} \in {}^{J_1\times J_2\times\ldots\times J_M}$ such that $I_n = J_m$, is defined as $\mathcal{C} = \mathcal{A}\,{}_n\times_m \mathcal{B}$. It has entries

$$\mathcal{C}(i_1,\ldots,i_{n-1},\ldots,i_N,j_1,\ldots,j_{m-1},j_{m+1},\ldots,j_M) = \sum_{i_n=1}^{I_n} \mathcal{A}(i_1,\ldots,i_{n-1},i_n,i_{n+1},\ldots,i_N)\mathcal{B}(j_1,\ldots,j_{m-1},i_n,j_{m+1},\ldots,j_M)$$

Note that this operation is different from the tensor product. It is like the n-mode product, except now a mode m also has to be specified since we are indexing over two tensors.

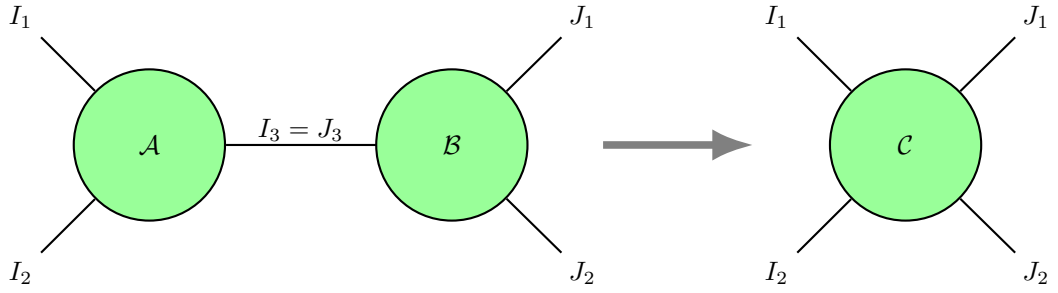


Figure 3: ${}_n\times_m$-product

Example

$$\text{Let } \mathbf{A} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

$$\mathbf{A}\,_1\times_1\,\mathbf{b} = \begin{bmatrix} 1(5)+2(6) \\ 3(5)+4(6) \end{bmatrix} = \begin{bmatrix} 5+12 \\ 15+24 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

2.5 Matricization

Let $\mathcal{A} \in {}^{I_1\times\ldots\times I_N}$ be an N-th order tensor. The k-th matricization of $\mathcal{A}$ is a matrix $\mathbf{A}_k \in {}_{I_1 I_2,\ldots,I_k\times I_{k+1}\times I_{k+2}\times\ldots\times I_N}$, whose elements are taken column-wise from A. The elements are $\mathbf{A}_{(k)}(i_1\ldots,i_k,i_{k+1},\ldots.,i_N) = \mathcal{A}(i_1,\ldots,i_k,i_{k+1},\ldots,i_N)$. This operation allows us to collapse a tensor into a matrix. However, this loses information about how the dimensions are related. Another name for it is unfolding.

Example

The slices of A along the 3rd mode are two 4×3 matrices.

$$A_1 = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 13 & 17 & 21 \\ 14 & 18 & 22 \\ 15 & 19 & 23 \\ 16 & 20 & 24 \end{bmatrix}.$$

Let us say we want to matricize along the first mode. This compresses the second and third mode into the columns of a matrix, so we would expect to obtain a matrix $\in \mathbb{R}^{4\times 6}$. We will do this by moving the values in the third mode into the columns.

The notation for matricization along the first mode is $A_{(1)}$.

$$A_{(1)} = \begin{bmatrix} 1 & 5 & 9 & 13 & 17 & 21 \\ 2 & 6 & 10 & 14 & 18 & 22 \\ 3 & 7 & 11 & 15 & 19 & 23 \\ 4 & 8 & 12 & 16 & 20 & 24 \end{bmatrix}$$

2.6 Frobenius norm

Let $\mathcal{A} \in {}^{I_1\times I_2\times\cdots\times I_N}$ be a tensor. The Frobenius norm of this tensor is defined as (Kolda and Bader, 2009)

$$\|\mathcal{A}\| = \sqrt{\sum_{i_1=1}^{I_1}\sum_{i_2=1}^{I_2}\cdots\sum_{i_N=1}^{I_N} a(i_1,i_2,\ldots i_N)^2}.$$

This notation means that we take every value in the tensor, square them, add them up, and then take the square root of that sum. Another way to think about this is vectorizing the tensor, squaring the elements, adding them up, and then taking the square root.

2.7 Existing tensor packages

There are currently several major R packages that implement tensor operations. The package **rTensor** creates an S4 class called Tensor. This is a wrapper of the array class, adding several methods and attributes. The major methods on a Tensor class include functions for computing the Frobenius norm, inner product, matricization, and tensor times matrix products. The package also facilitates several major tensor decompositions, including

canonical polyadic decomposition, general Tucker decomposition, multilinear principal component analysis, higher-order singular value decomposition, tensor singular value decomposition, and population value decomposition.

The package **tensorr**'s goal is to manipulate and store sparse tensors. It provides standard operations like multiplication and unfolding. It has two S4 class objects: dtensor and stensor. The stensor object stores the indices and values of non-zero elements, which provides a more efficient representation for sparse tensors.

The package **ttTensor** provides methods for the Tensor-Train decomposition. The three functions it provides are TTCross, TTSVD, and TTWOPT. The function TTCross decomposes the tensor using the skeleton decomposition algorithm while TTSVD uses the SVD and TTWOPT uses gradient descent.

The **tensormodels** package complements this existing software by focusing on statistical distributions for tensor-valued observations. It also uses base R arrays and lists of arrays so that users do not need to convert their data to a separate S4 tensor representation.

2.8 Tensor variate distributions

There are several tensor variate distributions that are generalizations of matrix variate distributions. This section introduces the distributions that are used by **tensormodels**.

Tensor variate normal

The tensor variate normal distribution can be considered a generalization of the multivariate and matrix variate normal distributions.

Let $\mathcal{Z}$ be an order-D tensor with IID standard normal entries and $\mathbf{\Sigma}_1, \ldots, \mathbf{\Sigma}_D$ be positive definite matrices describing the variance-covariance structure along each mode. For each mode, choose $\mathbf{A}_j$ such that $\mathbf{\Sigma}_j = \mathbf{A}_j \mathbf{A}_j^T, \qquad j = 1, \ldots, D$.

Then we can define $\mathcal{X}$ as

$$\mathcal{X} = \mathcal{M} + \mathcal{Z} \times_1 \mathbf{A}_1 \times_2 \mathbf{A}_2 \cdots \times_D \mathbf{A}_D.$$

This ensures that

$$\text{vec}(\mathcal{X}) \sim N\left(\text{vec}(\mathcal{M}), \mathbf{\Sigma}_D \otimes \cdots \otimes \mathbf{\Sigma}_2 \otimes \mathbf{\Sigma}_1\right)$$

Thus, the tensor variate normal is obtained by transforming a tensor of IID standard normal draws along each mode

If $\mathcal{X}$ has dimensions $n_1 \times \cdots \times n_D$, then

$$\text{vec}(\mathcal{X}) \sim N_{n^*}\left(\text{vec}(\mathcal{M}), \bigotimes_{d=1}^{D} \Sigma_d\right), \qquad n^* = \prod_{d=1}^{D} n_d.$$

The density function of the multilinear normal distribution is (Ohlson et al., 2013)

$$f(\mathcal{X}) = (2\pi)^{-n^*/2} \left(\prod_{d=1}^{D} |\Sigma_d|^{-n^*/(2n_d)}\right) \exp\left\{-\frac{1}{2} \text{vec}(\mathcal{X} - \mathcal{M})^T \left(\bigotimes_{d=1}^{D} \Sigma_d^{-1}\right) \text{vec}(\mathcal{X} - \mathcal{M})\right\}.$$

Each Σ_d is positive definite and describes the covariance structure corresponding to mode d.

Skewed distributions as transformations of the tensor variate normal

To generate tensor variate skewed distributions, we use the normal variance mean mixture model (Barndorff-Nielsen et al., 1982). An r-dimensional random vector x is a normal variance-mean mixture with mixing distribution F if, for a given $u \geq 0$ that follows a

Table 1: Mixing distributions used to construct the skewed tensor variate distributions.

Distribution	Distribution of W	Parameters	Parameter support
Tensor variate skewed-t	Inv-Gamma$(\nu/2, \nu/2)$	ν	$\nu > 0$
Tensor variate generalized hyperbolic	Gen Inv Gaussian$GenInvGaussian(\omega, 1, \lambda)$	λ, ω	$\omega > 0,\ \lambda \in \mathbb{R}$
Tensor variate variance gamma	Gamma(γ, γ)	γ	$\gamma > 0$
Tensor variate normal inverse Gaussian	Inv Gaussian$(1, \kappa)$	κ	$\kappa > 0$

probability distribution F on $[0, \infty)$, $x|u \sim N_r(\mu + u\beta, u\Sigma)$. $\Sigma \in \mathbb{R}^{r \times r}$ is a constant, positive-definite matrix and $\mu \in \mathbb{R}^r$ and $\beta \in \mathbb{R}^r$ are constant vectors.

We state that x is a normal variance-mean mixture with position μ, drift β, structure matrix Σ, and mixing distribution F. Different mixing distributions lead to different distributions. For example, if F, the mixing distribution, is the generalized inverse Gaussian distribution, then the resulting distribution of x is a generalized hyperbolic distribution.

In the tensor variate case, the normal variance mean mixture model is an efficient way to introduce skewness. A random tensor $\mathcal{X}$ can be written as

$$\mathcal{X} = \mathcal{M} + W\mathcal{A} + \sqrt{W}\mathcal{V}, \tag{1}$$

where $\mathcal{M}$ is a location tensor, $\mathcal{A}$ is a skewness tensor, $\mathcal{V} \sim \mathcal{N}_{n^*}(\mathcal{O}, \bigotimes_{d=1}^{D} \Sigma_d)$ and $W > 0$ is a positive random variable. The distribution of $\mathcal{X}$ will depend on the distribution of W.

The mixing distributions and their parameters for each of the skewed tensor variate distributions are summarized below. In each case, the support of W is $(0, \infty)$.

The tensor variate skewed-t distribution can be generated by setting $W \sim$ Inv-Gamma$(\nu/2, \nu/2)$ in Equation 1. We then say that $\mathcal{X} \sim \text{TVST}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^{D} \Sigma_d, \nu)$.

The tensor variate generalized hyperbolic distribution can be generated by setting $W \sim I(\omega, 1, \lambda)$ in Equation 1. We then say that $\mathcal{X} \sim \text{TVGH}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^{D} \Sigma_d, \lambda, \omega)$.

The tensor variate variance gamma distribution can be generated by setting $W \sim$ Gamma(γ, γ) in Equation 1. We then say that $\mathcal{X} \sim \text{TVVG}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^{D} \Sigma_d, \gamma)$.

The tensor variate normal inverse Gaussian distribution can be generated by setting $W \sim \text{IG}(1, \kappa)$ in Equation 1. We then say that $\mathcal{X} \sim \text{TVNIG}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^{D} \Sigma_d, \kappa)$.

These choices of W give the four skewed families implemented in **tensormodels**. Their common representation also makes it possible to use a shared estimation framework while changing the distribution of the latent mixing variable.

3 Package design

The main goal of tensortools is to provide functions for random number generation, density computation, and MLE estimation for common tensor variate distributions. Additionally, the package provides diagnostic tests for the structure of input data.

One intentional decision was to store independent, identically distributed draws from a tensor variate distribution as a list. For example, if we generate $n = 100$ draws from a tensor variate normal distribution of dimension $2 \times 3 \times 4$, the resulting R object will be a list of length 100. Each element in the list is a base R array object of dimension $2 \times 3 \times 4$. Other packages in this space created unique S3 and S4 class objects to store these types of draws. However, these choices are difficult for the average R user, as they have unique interactions and functions.

Another option would be to output a tensor of size $100 \times 2 \times 3 \times 4$; that is, create a tensor of one higher dimension where the first mode represents the index of the draw. The choice

to have the first mode rpresent the draw is arbitrary and could potentially confuse users, especially if they permute the dimensions.

```
three_dim_iid <- rtnorm(n = 100, mu = array(1:12, dim = c(2, 3, 4)))

length(three_dim_iid)
#> [1] 100
str(three_dim_iid[1:2], max.level = 1)
#> List of 2
#>  $ : num [1:2, 1:3, 1:4] 1.52 3.22 2.57 3.85 6.34 ...
#>  $ : num [1:2, 1:3, 1:4] -0.0797 1.5062 3.4586 3.817 5.5527 ...
#>  - attr(*, "dim")= int 2
```

Describe the package's data structures, model classes, and computational approach. Emphasize design decisions that matter to an R user.

3.1 Core workflow

This package contains random number generations for several tensor variate distributions. Below is a simulation from a tensor variate normal of size $2 \times 3 \times 4$. The dimension of the draws is the same as the mu object provided. The covariance structure is specified with a list of matrices, where each matrix must be a square of the corresponding mode.

```
mu_true <- array(1:24, dim = c(2, 3, 4))

S1 <- crossprod(matrix(rnorm(4), nrow = 2))
S2 <- crossprod(matrix(rnorm(9), nrow = 3))
S3 <- crossprod(matrix(rnorm(16), nrow = 4))

sigmas_true <- list(S1, S2, S3)

tvn_draws <- rtnorm(n = 1e3, mu = mu_true,
                    sigmas = sigmas_true)
```

Given a list of draws, the package can estimate the MLEs using the function `tensor_mle()`. The model argument is required to know which distribution to fit to the data.

```
tvn_mles <- tensor_mle(tvn_draws, model = "normal")

str(tvn_mles)
#> List of 6
#>  $ mu    : num [1:2, 1:3, 1:4] 0.838 2.514 2.803 4.669 4.983 ...
#>  $ sigmas:List of 3
#>   ..$ : num [1:2, 1:2] 0.277 -0.383 -0.383 1.723
#>   ..$ : num [1:3, 1:3] 1.389 1.416 0.119 1.416 1.597 ...
#>   ..$ : num [1:4, 1:4] 17.53 3.23 3.36 4.64 3.23 ...
#>  $ loglik: num -8984
#>  $ k     : num 41
#>  $ AIC   : num 18050
#>  $ BIC   : num 18252
```

There are also density functions for each distribution. For the tensor variate normal, the required arguments are mu and sigmas. There is an optional log argument to compute the log density rather than the density itself.

```
dtnorm(tvn_draws[[1]], mu = mu_true, sigmas = sigmas_true)
#> [1] 0.001097981
```

3.2 Tensor decompositions

The idea of decompositions can also apply to tensors. The package provides the Tucker, PARAFAC, higher-order singular value, and Tensor-Train decompositions.

Tucker decomposition

The idea behind the Tucker decomposition is similar to rank factorization for matrices. The Tucker decomposition factorizes an N-th order tensor into a core tensor and a list of matrices (Hitchcock, 1927):

$$\mathcal{A} = \mathcal{B} \times_1 \mathbf{U}_1 \times_2 \mathbf{U}_2 \cdots \times_N \mathbf{U}_N.$$

The core tensor $\mathcal{B}$ has the same order as the original tensor $\mathcal{A}$, and the number of $\mathbf{U}$ matrices is also the number of modes. The modes of $\mathcal{B}$ are called the Tucker ranks, which determine how reduced the form is.

Each matrix $\mathbf{U}_n$ contains principal factors for mode n. The $\mathbf{U}_n$ is a basis for the nth mode, and thus spans a subspace of $\mathbb{R}^{I_n}$. The core tensor $\mathcal{B}$ represents the strength of interactions between factors. The idea behind this decomposition is to reduce the dimensionality of the input. We are able to summarize the higher-order object $\mathcal{A}$ with a smaller core tensor $\mathcal{B}$ and several factor matrices $\mathbf{U}_1, \mathbf{U}_2, \ldots, \mathbf{U}_N$. However, one key issue with the Tucker decomposition is that it is not unique.

PARAFAC decomposition

The PARAFAC decomposition has several other names—CANDECOMP, CP, and Tensor Rank decomposition. PARAFAC stands for parallel factor analysis while CANDECOMP stands for canonical polyadic decomposition. This decomposition is a special case of the Tucker decomposition, where the core tensor is superdiagonal.

Let us say we want to approximate the tensor $\mathcal{A} \in {}^{I_1 \times I_2 \times \cdots \times I_N}$ with a linear combination of R rank 1 tensors. The approximation can be written as

$$\mathcal{A} \approx \sum_{r=1}^{R} \lambda(r) \mathbf{u}_1(:, r) \otimes \cdots \otimes \mathbf{u}_N(:, r).$$

However, determining the rank of a tensor is an NP hard problem, so it is difficult to determine the value of R necessary to approximate the original tensor. The package computes this decomposition using the alternating least squares algorithm described by Carroll and Chang (1970) and Harshman (1970).

Higher-order singular value decomposition

The higher-order singular value decomposition (HOSVD) is a specific case of the Tucker decomposition with restrictions. The matrices $\mathbf{U}$ are unitary and the core tensor $\mathcal{B}$ has subtensors, defined as setting the nth index to a value α, that are orthogonal (De Lathauwer et al., 2000). Additionally, the subtensors have a descending order of Frobenius norms:

$$\|\mathcal{B}_{i_n=1}\| \geq \|\mathcal{B}_{i_n=2}\| \geq \cdots \geq \|\mathcal{B}_{i_n=I_n}\| \geq 0.$$

These Frobenius norms are the n-mode singular values of $\mathcal{A}$, and the vector $U_i(n)$ is the ith n-mode singular vector.

Tensor-Train decomposition

The Tensor-Train decomposition factorizes an N-th order tensor $\mathcal{A}$ in a product of third-order tensors (Oseledets, 2011). These are denoted as $\mathbf{U}_1, \mathcal{U}_2, \ldots, \mathbf{U}_N$ and are called TT-cores:

$$\mathcal{A}(i_1, \ldots, i_N) = \mathbf{U}_1(i_1, :)\mathcal{U}_2(:, i_2, :)\mathcal{U}_3(:, i_3, :) \ldots \mathbf{U}_N(:, i_N).$$

The package computes this decomposition using the TT-SVD algorithm. The user can provide the ranks directly or allow them to be selected using an approximation accuracy.

4 Model fitting and inference

One major functionality that tensormodels provides is MLE estimation. This is done via two different algorithms: for the tensor variate normal distribution, a flip-flop algorithm is used. For tensor variate skewed distributions, an EM algorithm is used.

Before discussing the tensor variate normal, we discuss the simpler matrix variate normal distribution. The matrix variate normal distribution can be considered a special case of the multivariate normal distribution, where we force the restriction that the covariance matrix is separable. That is, the covariance of the vectorized matrix draws can be described as the Kronecker product of two matrices.

Let $\mathbf{X} \sim N(\mathbf{M}, \mathbf{U}, \mathbf{V}) \in \mathbb{R}^{n \times p}$ and set $\boldsymbol{y} = \text{vec}(\mathbf{X})$. The matrix variate normal distribution imposes the separable covariance restriction

$$\boldsymbol{\Sigma} = \mathbf{V} \otimes \mathbf{U},$$

where $\mathbf{U}$ is the row covariance matrix and $\mathbf{V}$ is the column covariance matrix. Therefore

$$\text{vec}(\mathbf{X}) \sim N_{np}\left(\text{vec}(\mathbf{M}), \mathbf{V} \otimes \mathbf{U}\right).$$

This is a nonlinear restriction on the natural parameter space of the full multivariate normal family. For the matrix variate normal, the covariance matrix is restricted to be defined as the Kronecker product of two covariance matrices. This makes its parameter space a lower-dimensional curved subset of the full multivariate normal distribution (Roś et al., 2016).

By default, the initial values for the parameters are chosen in different ways. The location parameter $\mathcal{M}$ is started as the median of the draws. This is because the mean of the draws is dependent on the skew and parameters. Each covariance matrix is initialized by centering by the median, matricizing along the corresponding dimension, and adding up the scatter matrix across all of the draws. This total is normalized to have a trace equal to its dimension to ensure the scale is not disproportionate. Initial values for the parameters are set to be conservative and specific to the distribution. For example, for the skewed t, $\nu = 20$. Finally, the skew parameter $\mathcal{A}$ is computed as a function of the expected value of the draws, $\mathcal{M}$, and the relevant parameters for that distribution.

The individual covariance matrices in a Kronecker product are not defined uniquely. Multiplying one covariance matrix by a constant and another by the inverse of that constant produces the same full covariance matrix. The package addresses this by normalizing the trace of each covariance matrix except the final one to equal its corresponding mode dimension. This preserves the overall Kronecker covariance while giving the individual mode covariance estimates a consistent scale.

For the matrix variate normal model, differentiating the likelihood gives the updates

$$\hat{\mathbf{V}} = \frac{1}{Np} \sum_{i=1}^{N} \mathbf{X}_{ic}^T \hat{\mathbf{U}}^{-1} \mathbf{X}_{ic}$$

and

$$\hat{\mathbf{U}} = \frac{1}{Nn} \sum_{i=1}^{N} \mathbf{X}_{ic} \hat{\mathbf{V}}^{-1} \mathbf{X}_{ic}^{T}.$$

Each estimate depends on the other, which leads to the iterative flip-flop algorithm. If $N > \max(n, p)$, then the flip-flop algorithm has only one solution and the MLEs are unique under the conditions described by Srivastava et al. (2009).

Convergence checks are done by comparing the change in likelihood relative to the previous iteration. At the t-th iteration, the equation used is

$$A = \frac{\text{loglik}_t - \text{loglik}_{t-1}}{\text{loglik}_{t-1}}$$

The algorithm ends when $A < \epsilon$, and by default $\epsilon = 1e - 6$.

Explain estimation, initialization, convergence checks, uncertainty quantification, and any constraints or identifiability conventions.

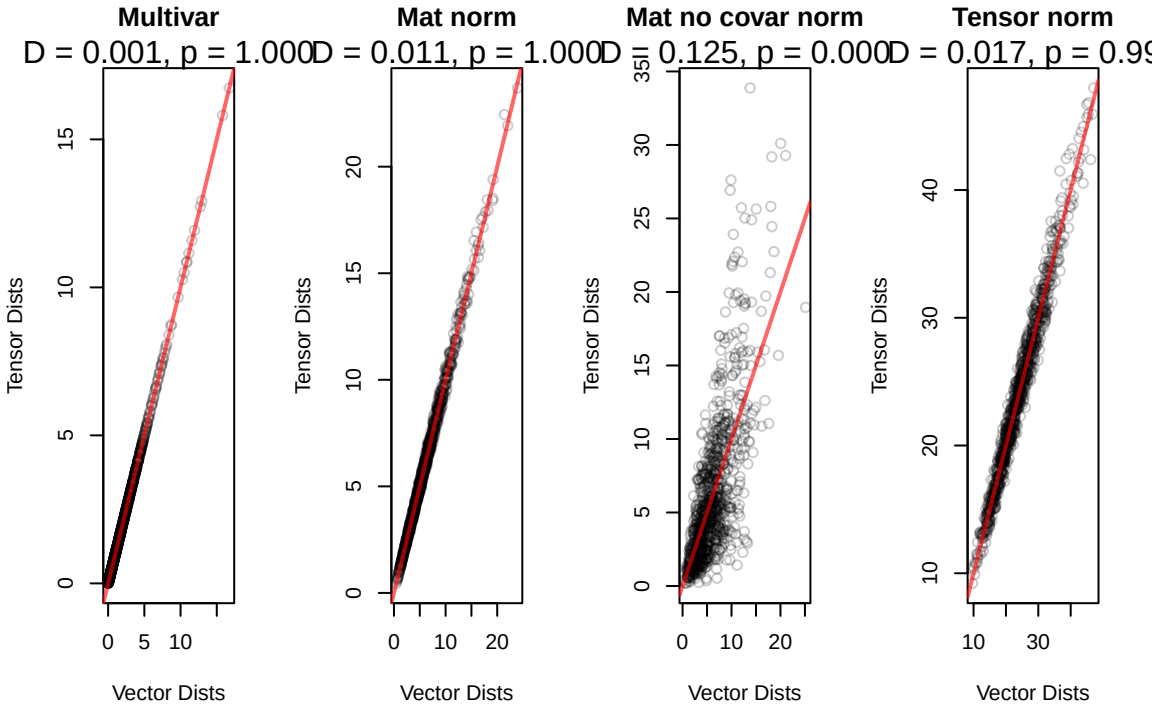
5 Diagnostic Tests

We assess tensor variate normality using the Mahalanobis distance. If $\mathcal{X}$ follows an order-O tensor variate normal distribution, then $\text{vec}(\mathcal{X})$ follows a multivariate normal distribution with mean $\text{vec}(\mathcal{M})$ and covariance matrix $\mathbf{\Sigma}_O \otimes \cdots \otimes \mathbf{\Sigma}_1$. This is why we can vectorize the tensor draws and compare the multivariate Mahalanobis distances to the tensor variate Mahalanobis distances. If we have properly reconstructed the structure using the MLEs $\hat{\mathcal{M}}$ and $\hat{\mathbf{\Sigma}}_O, \cdots \hat{\mathbf{\Sigma}}_1$, then the standardized draws should be from a standard normal.

If a tensor normal covariance structure is present, then the multivariate and tensor variate distances should be close to one another. Following the matrix variate normality paper, this can be visualized with a distance-distance plot: when the tensor normal structure is appropriate, the distances lie close to the reference line; when the Kronecker product structure is absent, the distances diverge.

The function `mahalanobis_dist()` computes both distances using fitted MLEs, and `mahalanobis_test()` performs a two-sample Kolmogorov-Smirnov test to compare their empirical distributions.

The figure below shows the Mahalanobis distance-distance plots along with the KS test statistic and p-value. From left to right, the input data is a multivariate normal, matrix variate normal with a separable covariance structure, matrix variate with nonseparable covariance matrices, and tensor variate normal with a separable covariance structure.



We are also interested in performing a likelihood ratio test to see if one of the covariance matrices might be the identity matrix. Thus, we run the algorithm and restrict these matrices.

For a restriction set $\mathcal{R} \subset \{1, \ldots, D\}$, the restricted model fixes $\Sigma_r = I_{n_r}$ for every $r \in \mathcal{R}$, while all other covariance matrices are estimated. The likelihood ratio test compares

$$H_0 : \Sigma_r = I_{n_r} \; \forall r \in \mathcal{R} \quad \text{vs.} \quad H_A : \Sigma_r \neq I_{n_r} \text{ for at least one } r \in \mathcal{R}.$$

The test statistic is

$$2\{\ell(\hat{\theta}_{\text{full}}) - \ell(\hat{\theta}_{\mathcal{R}})\} \sim \chi^2_{\nu_{\mathcal{R}}}, \quad \nu_{\mathcal{R}} = \sum_{r \in \mathcal{R}} \left\{ \frac{n_r(n_r+1)}{2} - 1 \right\}$$

If the p-value is large, then we fail to reject the null hypothesis and conclude the restricted model fits the data as well as the unrestricted model. If the p-value is small, we reject the null hypothesis, which states the unrestricted model has a significantly better fit than the restricted model.

As expected, all of the restrictions lead to a model that is significantly different from the full model. All the covariance matrices are not the diagonal to best model the draws.

```
res <- map_dfr(list(NULL, c(1), c(2), c(3), c(1, 2), c(1, 3), c(2, 3)),
        draws = tvn_draws, test_restrict)

knitr::kable(
  res,
  digits = 3,
  escape = FALSE,
  booktabs = TRUE,
  caption = "LRT for Full Tensor Variate")
```

Now, we generate data where the second covariance matrix is diagonal. The model that restricts the 2nd covariance to be diagonal has a $p - val > 0.05$, meaning it is does not have a significantly smaller likelihood than the full unrestricted model.

Table 2: LRT for Full Tensor Variate

restrict	statistic	df	p-val
None	0.00	0	1
1	13257.02	2	0
2	61541.52	5	0
3	163633.38	9	0
1, 2	75108.59	7	0
1, 3	178430.32	11	0
2, 3	226430.11	14	0

Table 3: LRT for Second Sigma Identity

restrict	statistic	df	p-val
None	0.000	0	1.000
1	12103.247	2	0.000
2	9.138	5	0.104
3	47443.558	9	0.000
1, 2	12119.269	7	0.000
1, 3	59436.610	11	0.000
2, 3	47461.651	14	0.000

6 Performance and validation

6.1 Parameter recovery

To validate the distributions, we can compare the MLEs to the true values used to generate draws.

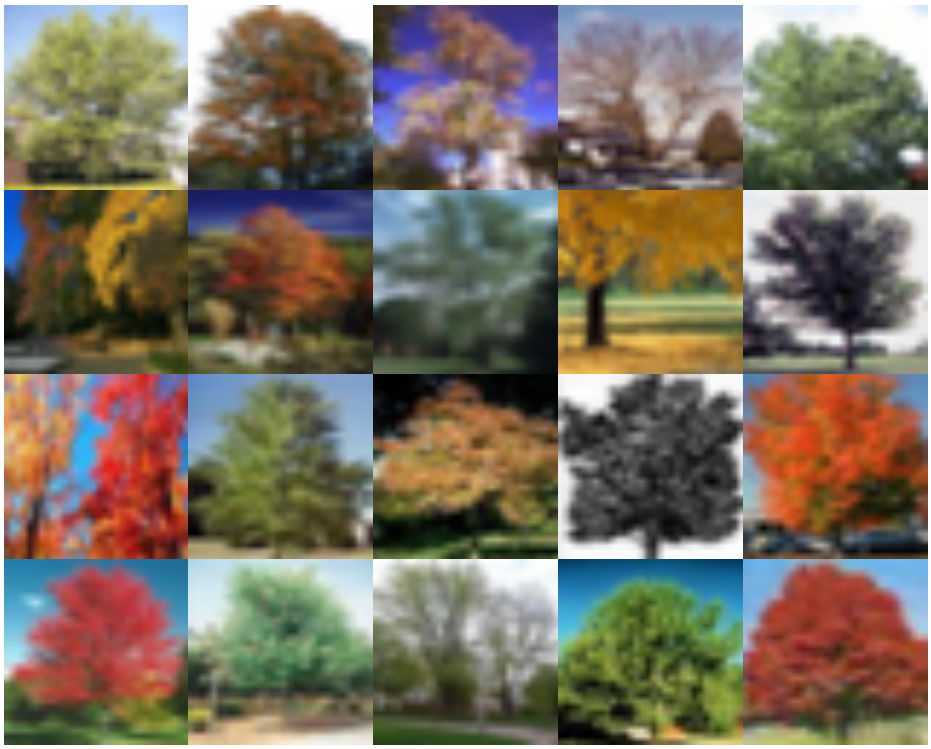
```
mean((tvn_mles$mu - mu_true)^2)
#> [1] 0.04388826

for(i in 1:3) {
  true_scaled <- sigmas_true[[i]] / sum(diag(sigmas_true[[i]]))
  est_scaled <- tvn_mles$sigmas[[i]] / sum(diag(tvn_mles$sigmas[[i]]))

  print(mean((est_scaled - true_scaled)^2))
}
#> [1] 1.348412e-06
#> [1] 2.30351e-06
#> [1] 7.609639e-06
```

7 Examples

Now that we have set up our tensor distributions, we can use them on a real-world example. The CIFAR 100 dataset contains images of a set of 100 common objects. In this case, we will only be using the maple trees, which have an ID value of 48.

Once we have the CIFAR dataset loaded in from torchvision, we can plot 20 example images. The images are stored as an array of size 32 x 32 x 3. This is an image that is 32 x 32 pixels, and the last mode represents the red, green, and blue color values.

his is an image that is 32 x 32 pixels stored as red, green, and blue values. The values vary from 0 to 255 and represent the intensity of that color.



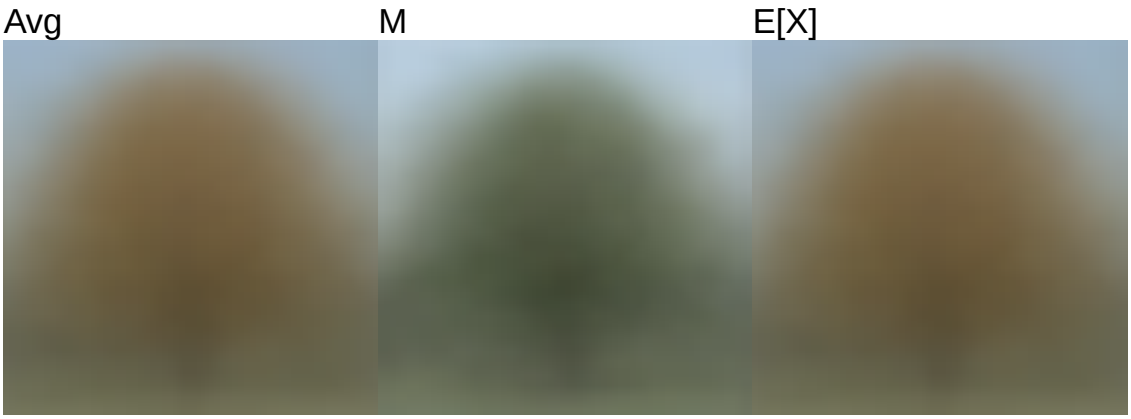
We can fit the best model to this set of images. The function `model_compare()` takes in a dataset and fits all of the possible tensor models. It returns the result as a data frame with columns containing the name of the model, log likelihood, number of parameters denoted as k, AIC, BIC, and the model object itself. We can then select the model with the lowest BIC to proceed.

```
model_compare <- function(draws) {
  res_normal <- tensor_mle(draws, model = "normal", quiet = FALSE)
  res_skewt <- tensor_mle(draws, model = "skewt", quiet = FALSE)
  res_vargam <- tensor_mle(draws, model = "vargamma", quiet = FALSE)
  res_invgauss <- tensor_mle(draws, model = "invgauss", quiet = FALSE)
  res_genhyper <- tensor_mle(draws, model = "genhyper", quiet = FALSE)

  all_mod <-
    tibble(model = c("Normal", "Skewt", "Vargamma", "Invgauss", "Genhyper"),
    loglik = c(res_normal$loglik, res_skewt$loglik,
               res_vargam$loglik, res_invgauss$loglik, res_genhyper$loglik),
    k = c(res_normal$k, res_skewt$k,
          res_vargam$k, res_invgauss$k, res_genhyper$k),
    AIC = c(res_normal$AIC, res_skewt$AIC,
            res_vargam$AIC, res_invgauss$AIC, res_genhyper$AIC),
    BIC = c(res_normal$BIC, res_skewt$BIC,
            res_vargam$BIC, res_invgauss$BIC, res_genhyper$BIC),
   mod_obj = list(res_normal, res_skewt, res_vargam, res_invgauss, res_genhyper)
  )

  all_mod
}

#res <- model_compare(maple_trees)

#res |> select(model, loglik, BIC)
```

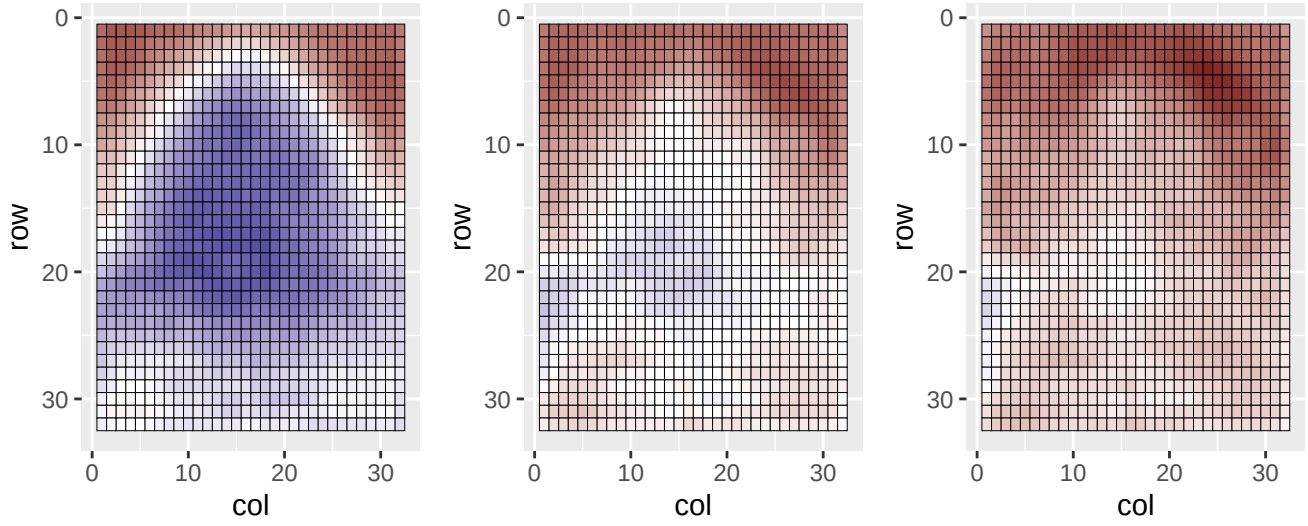
```
#best_mod <- res$mod_obj[[which.min(res$BIC)]]

best_mod <- tensor_mle(maple_trees, model = "invgauss", quiet = FALSE)
```

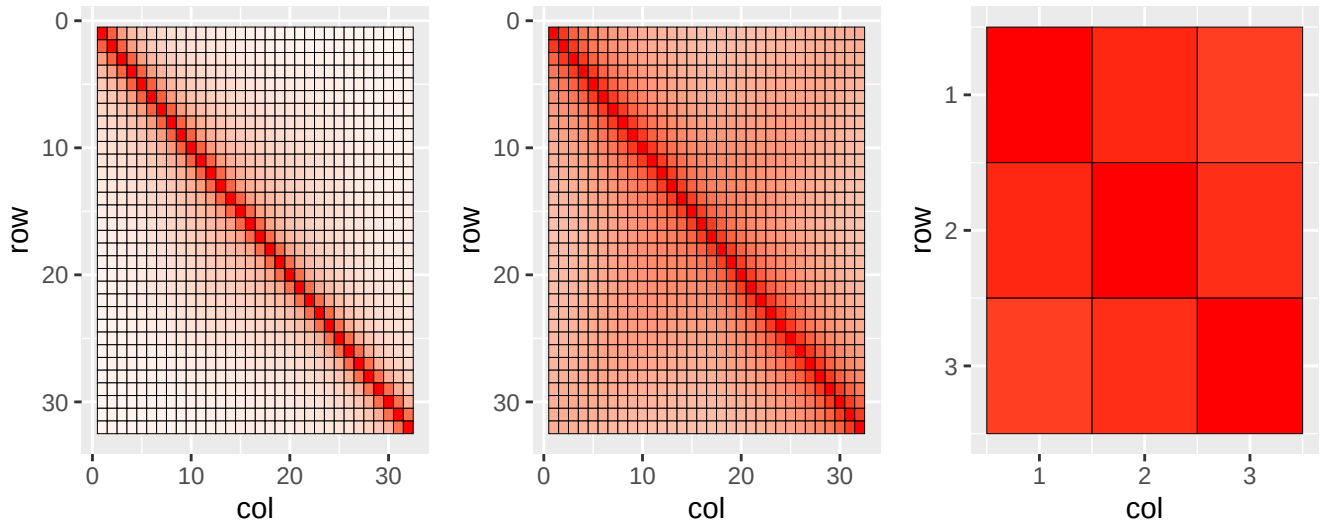
In this case, the best model was the normal inverse Gaussian. The expected value of its draws is $\mathcal{M} + \mathcal{A}/\kappa$, where $\mathcal{M}$ is the location parameter and $\mathcal{A}$ is the skew parameter. The plot below contains three images: the first is the empirical average of all of the maple trees without a model assumption. The second is the location parameter $\mathcal{M}$ from the model. Lastly, we have $E[X]$, which is the expected average from the model.

$\mathcal{M}$ is closer to a traditional green maple tree. This seems to be a good starting point to model the variety of colors of the maples, from green to red. Once we add the skew parameter, the $E[X]$ appears similar to the empirical average.

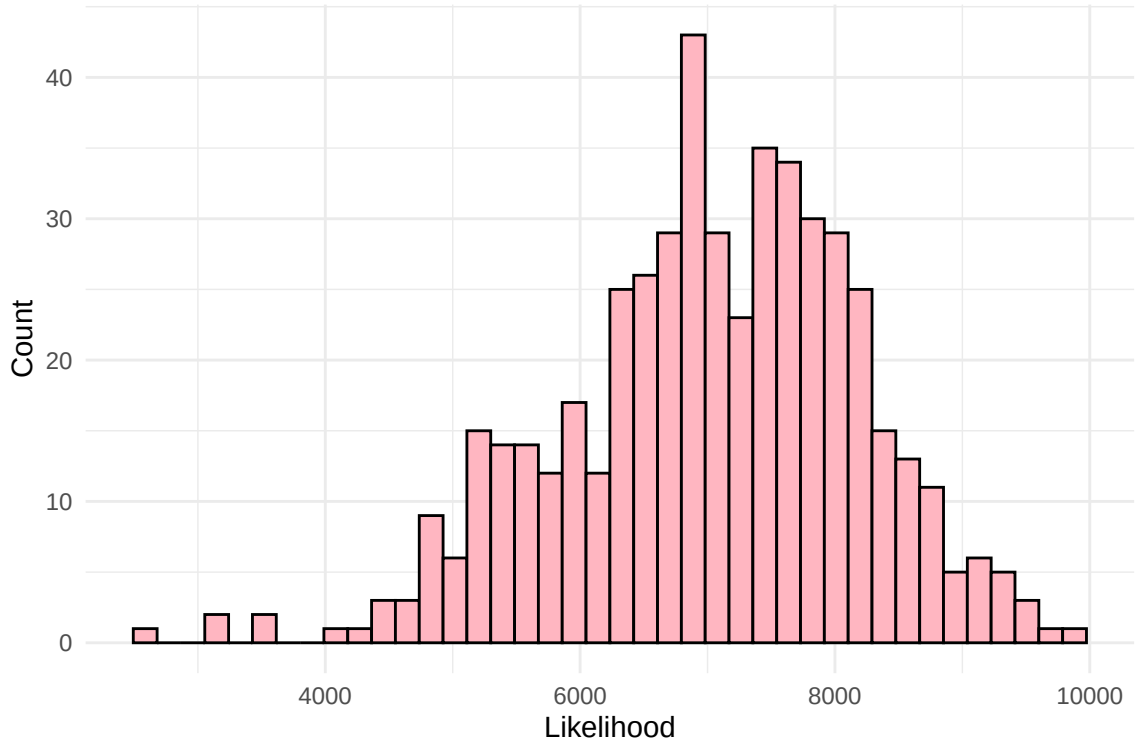


We can also create summary plots of the skew $\mathcal{A}$ and the covariance matrices Σ from the model. We converted the covariance matrices to correlation matrices to make it easier to interpret each matrix, since this places them on the same scale.



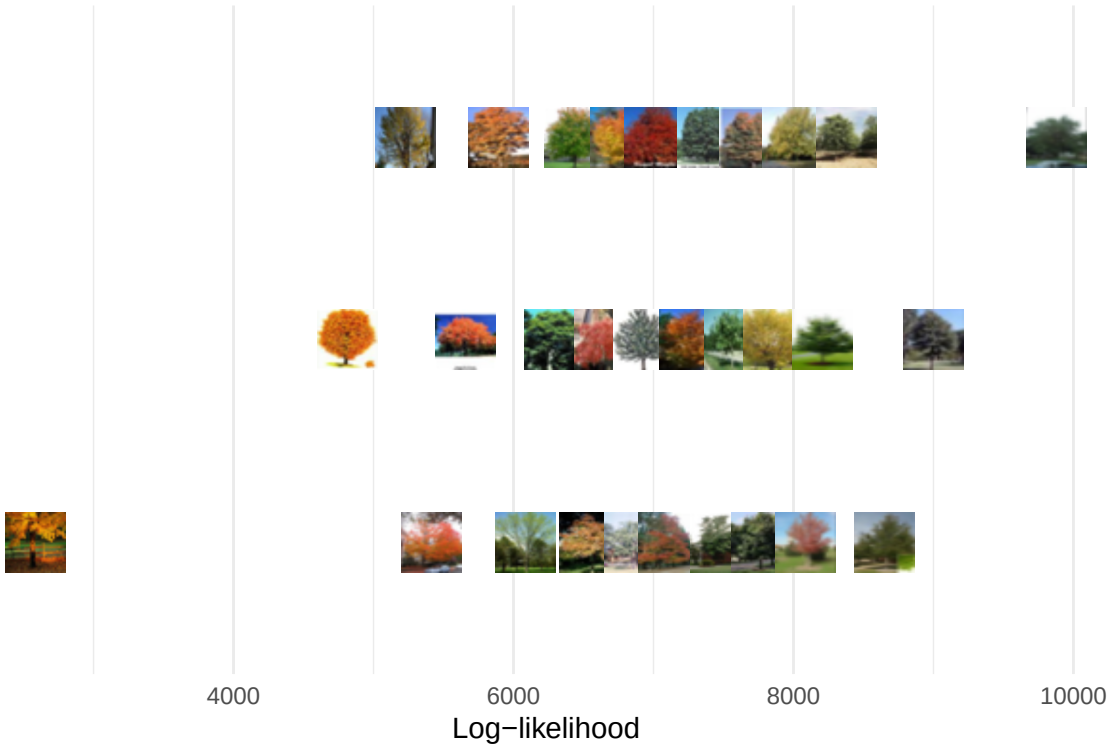


Now that we have the best model for the maple trees, we can obtain the likelihood of each image.



There are clearly some outliers, both at much smaller and much larger likelihoods. Out of curiosity, I plotted the lowest likelihoods. All of these maples are predominantly orange and red, which differ from the average tensor that the model had. It seems that the data contains many more green maples, so the model struggles to account for the possible variation in tree color.

As expected, all of the maples that had a large likelihood follow the pattern of being a forest green and mainly in the center of the image.



8 Discussion

Summarize the package's strengths, limitations, and intended scope. Mention important future work without turning this section into a feature wish list.

9 Reproducibility

List any external data and scripts required to reproduce the article. Place supporting code in `scripts/`, data in `data/`, and figure assets in `figures/`.

```
#> R version: 4.5.2
#> Platform: aarch64-apple-darwin20
#> rjtools version: 1.0.21
```

10 Acknowledgments

Add funding, contributor, and institutional acknowledgments here.

References

O. Barndorff-Nielsen, J. Kent, and M. Sørensen. Normal variance-mean mixtures and z distributions. *International Statistical Review*, 50(2):145–159, 1982. [p5]

J. D. Carroll and J.-J. Chang. Analysis of individual differences in multidimensional scaling via an n-way generalization of eckart-young decomposition. *Psychometrika*, 35(3):283–319, 1970. doi: 10.1007/BF02310791. [p8]

L. De Lathauwer, B. De Moor, and J. Vandewalle. A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21:1253–1278, 2000. doi: 10.1137/S0895479896305696. [p8]

R. A. Harshman. Foundations of the parafac procedure: Models and conditions for an explanatory multimodal factor analysis. 1970. [p8]

F. L. Hitchcock. The expression of a tensor or a polyadic as a sum of products. *Journal of Mathematics and Physics*, 6(1-4):164–189, 1927. doi: 10.1002/sapm192761164. [p8]

T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3): 455–500, 2009. doi: 10.1137/07070111X. [p4]

M. Ohlson, M. R. Ahmad, and D. von Rosen. The multilinear normal distribution: Introduction and some basic properties. *Journal of Multivariate Analysis*, 113:37–47, 2013. doi: 10.1016/j.jmva.2011.05.015. [p5]

I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5): 2295–2317, 2011. doi: 10.1137/090752286. [p9]

B. Roś, F. Bijma, J. C. de Munck, and M. C. M. de Gunst. Existence and uniqueness of the maximum likelihood estimator for models with a kronecker product covariance structure. *Journal of Multivariate Analysis*, 143:345–361, 2016. doi: 10.1016/j.jmva.2015.05.019. [p9]

M. S. Srivastava, T. von Rosen, and D. von Rosen. Estimation and testing in general multivariate linear models with kronecker product covariance structure. *Sankhyā: The Indian Journal of Statistics, Series A*, 71(2):137–163, 2009. [p10]

Anupama Kannan
Baylor University
Department of Statistics
Waco, Texas
ORCID: 0000-0003-2789-6997
Anupama_Kannan1@baylor.edu

David Kahle
Baylor University
Department of Statistics
Waco, Texas
ORCID: 0000-0002-9999-1558
David_Kahle@baylor.edu

Michael Gallaugher
Baylor University
Department of Statistics
Waco, Texas
ORCID: 0000-0002-5487-8965
Michael_Gallaugher@baylor.edu